```
$replacements = ['nimble ${1}'];
print preg_filter($patterns, $replacements, $text);
```

```
Output:
The  goldfish overtook the shark.
```

The first argument to the function can also be an array of strings, in which case all of them will be transformed in turn, equivalent to calling `preg_filter()` multiple times:

```
$text = [
    'The goldfish swam away from the shark.',
    'The shark ate the goldfish.'
];
$patterns = [
    '/(goldfish)/',
    '/(shark)/'
];
$replacements = [
    'sneaky ${1}',
    'naughty ${1}'
];
print_r(preg_filter($patterns, $replacements, $text));
```

```
Output:
Array
(
    [0] => The sneaky goldfish swam away from the naughty shark.
    [1] => The naughty shark ate the sneaky goldfish.
)
```

Note that the return value is now an array of transformed strings.

preg_replace(), preg_replace_callback(), preg_replace_callback_array()

`preg_replace()` is almost identical to `preg_filter()`: it has exactly the same parameters, and the only difference between the two is that `preg_replace()` will return the unchanged sample string(s) if no match is made: